Experiment No: 06

Title: Implement Stack using Linked List

Aim of Experiment:

Implement the Stack using link list and perform operations like INSERT, DELETE, DISPLAY.

Mapped to Course Objective:

To solve problems using data structures such as linear lists and stacks

Mapped to Course Outcome:

Implement stacks for various applications.

After performing this experiment, student should be able to understand the implementation difference between stack and queue. Also this will clear the implementation conceptsStacks.

Theory:

Stack:

A stack is an ordered collection of items into which items may be inserted and deleted from one end called Top of stack. The stack is a LIFO i.e. element which is put in the stack the last is the first to come out.

Static Implementation:

Since the stack is a ordered list of items we can implement stack using array. The static implementation using array will have limitations on size of stack as array size has to be predefined also the element is not physically deleted so that it does not release memory allotted for the element which has to be removed.

If TOP==-1 it indicates stack is empty.

If TOP==MAX-1 it indicates stack is full.

We shall also define the new operation called ‘init’ due to which stack will initialise the TOP to -1 to indicate an empty stack.

Dynamic implementation of stack:

Dynamic implementation of stack involves stack using Linked list.

In linked list:There is no limitation of size, memory being only

constraint.

i) Element will be physically deleted thus releasing the memory allotted for the element which has to be removed.]

ii) There is no leakage of memory.

iii) Since random access to any element is not required in a stack, the linked list representation is preferred over the sequential organisation.

In this implementation the stack will be maintained as a singly linked list.

The first node will be pointed to by a pointer called 'top'. Initially the stack is empty. Hence there will be no list and top contain NULL. Whenever an element has to be pushed, we will have to make a node to store the element and add this node to the beginning of list. Top will always point to recently added node. When the pop operation has to be performed, the first node will be deleted and the next node will become the new TOP.

Code:

```
#include<stdio.h>
#include<stdlib.h>

struct node{
   int data;
   struct node *next;
};

struct node *top=NULL;

void insert(int x){
   struct node *newnode;
   newnode=(struct node*)malloc(sizeof(struct node));
   newnode->data=x;
   newnode->next=top;
   top=newnode;
}

void delete(){
   struct node *temp;
   if(top==NULL){
      printf("Stack Underflow\n");
   }
```

```
    else{
        temp=top;
        printf("Deleted element: %d\n",top->data);
        top=top->next;
        free(temp);
    }
}

void display(){
    struct node *temp=top;
    if(top==NULL){
        printf("Stack is Empty\n");
        return;
    }
    while(temp!=NULL){
        printf("%d ",temp->data);
        temp=temp->next;
    }
    printf("\n");
}

int main(){
    insert(10);
    insert(20);
    insert(30);

    printf("Stack: ");
    display();

    delete();

    printf("After deletion: ");
    display();

    return 0;
}
```

Output:

```
Stack: 30 20 10
Deleted element: 30
After deletion: 20 10
```

M.M.C.O.E.
Page No: Date:

Assignment 6

Q1 What is Queue?
-> Queue is linear data str in which data can be inserted from rear end and deleted from front end.

Q2 How queue is represented?
-> Queue can be represented using array & L.L.

Q3 Enlist types of queues.
-> 1) linear queue
2) circular queue
3) priority queue

Q4 What is the concept of circular queue.
-> In a circular queue last node is connected back to first node to make a circle.

Q5 Applicatn of Queue.
-> 1) Categorizing the data
2) Queue Simulatn

Q6 What is Deque?
-> The deque is contracth of name double ended queue linear list in which elements can be added or removed at either end but not middle.